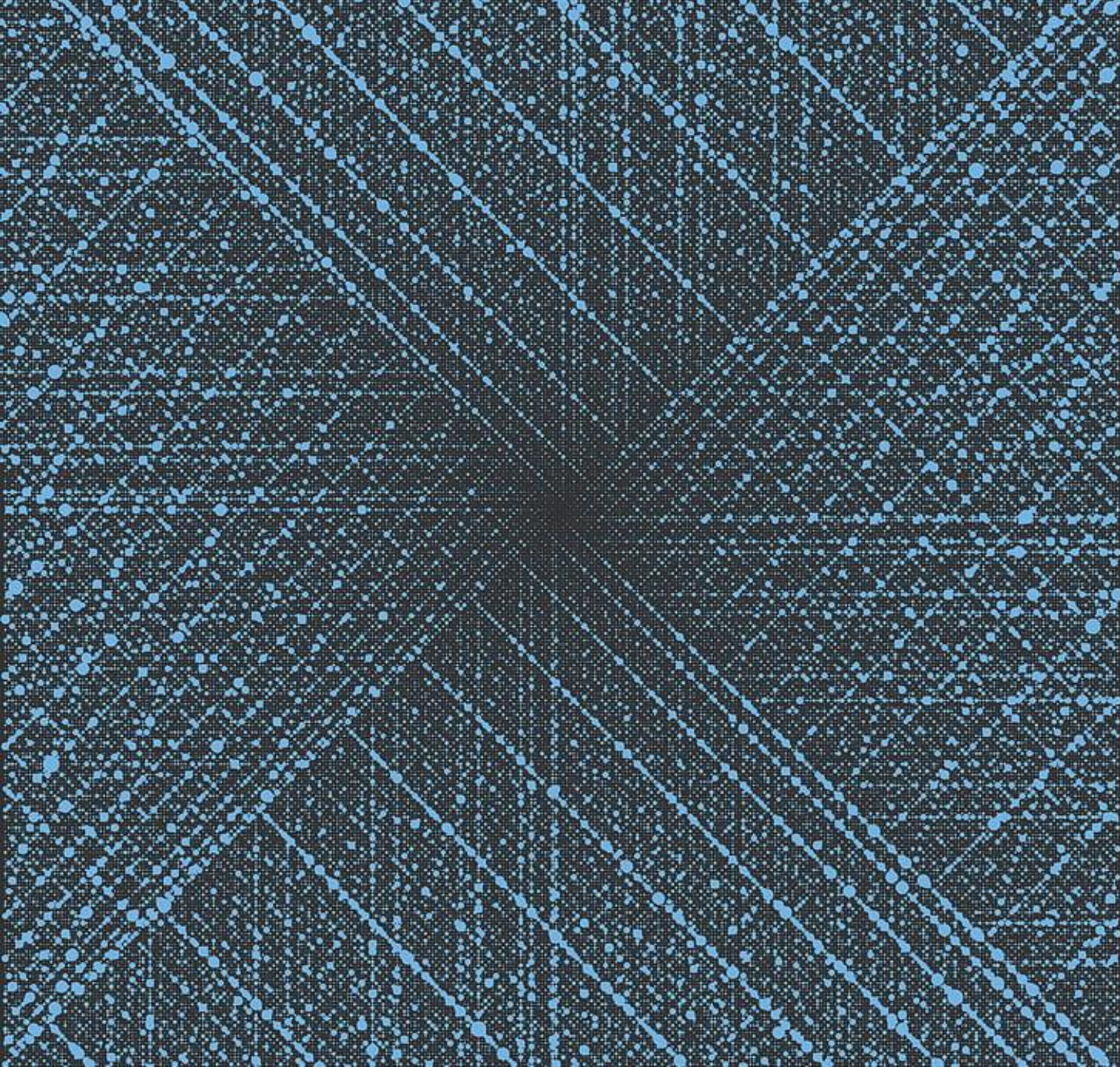


The left half of the image features a dark, textured background with a fine grid of small dots. Overlaid on this are numerous thin, light blue lines that intersect at various angles, creating a complex, web-like pattern. The lines vary in thickness and brightness, with some appearing as solid streaks and others as faint trails of dots.

Chapter 7

Mathematical **Background** **Part II**

Outlines

1. Prime and Composite Numbers.
2. Euler's and Fermat's Little Theorems.
3. Prime Numbers and Primality Testing.
4. Chinese Remainder Theorem.
5. Fast Exponentiation Algorithm.

Learning Objectives

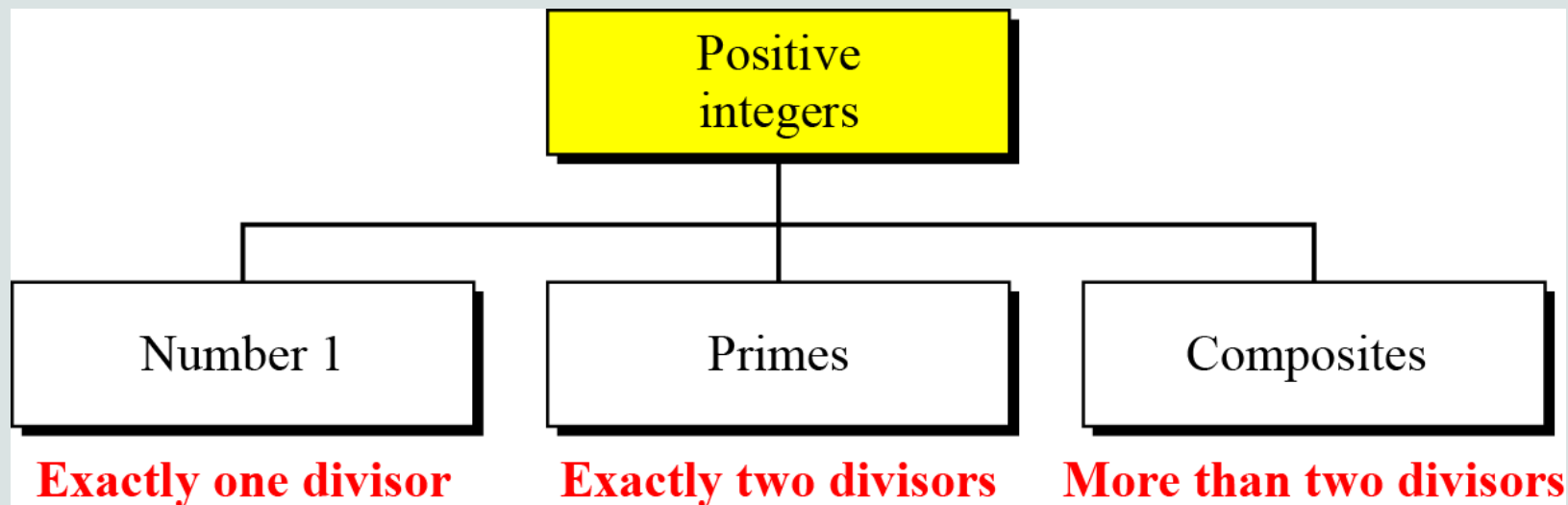
In the end of this Chapter 7, students will be able:

1. To understand the difference between **prime** and **composite numbers**.
2. To understand and apply the **Euler's** and **Fermat's Little Theorems**.
3. To **perform primality tests** on prime numbers.
4. To understand and apply the **Chinese Remainder Theorem** to solve **congruence equations**.
5. To understand and perform **fast exponentiation operation** using fast exponentiation algorithm.

PRIME AND COMPOSITE NUMBERS

Prime and Composite Numbers

- In general, positive integers can be divided into three groups:
 1. The number 1. (It has **exactly 1** divisor, which is 1 itself)
 2. **Prime** numbers. (It has **exactly 2** divisors, which are 1 and itself)
 3. **Composite** Numbers. (It has **more than 2** divisors)



Prime and Composite Numbers

❖ Definition 1:

*A positive integer is a **prime** if and only if it has exactly two divisors, 1 and itself. Otherwise, it is a **composite**.*

❖ Theorem 2:

*There are **infinitely many** prime numbers.*

Prime and Composite Numbers

- Examples of prime numbers less than 100.
- The **purple** coloured indicate the primes, while the rest are composite numbers
- **Note:**
 1. Number 2 is the **smallest** and the only **even prime** number.
 2. We will learn how to determine small prime numbers later.

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Prime and Composite Numbers

❖ For every positive integer (either prime or composite), there always exists an unique prime decomposition.

❖ **Theorem 3:**

Let $n \geq 2$ be an integer. Then this n can be decomposed into **prime factors** of $n =$

$$p_1^{e_1} \cdot p_2^{e_2} \cdot p_3^{e_3} \cdots p_r^{e_r}, \text{ with } p_i \text{ distinct primes, and positive exponents } e_i.$$

❖ **For examples:**

1. $100 = 2^2 \cdot 5^2$

2. $8645 = 5 \cdot 7 \cdot 13 \cdot 19$

Prime Numbers – Cardinality

- By **Theorem 2**, there are infinitely many prime numbers. To estimate the number of primes within a certain range, a function $\pi(n)$ can be used.
- This **function $\pi(n)$** estimate the **number of primes that is smaller than or equal to n** .

$$\frac{n}{\ln n} < \pi(n) < \frac{n}{\ln n - 1.08366}$$

➤ Examples:

1. $\pi(10) = 4$
2. $\pi(100) = 25$

Prime Numbers – Cardinality

❖ **Example 1:** Find the number of primes less than 1,000,000 (one million).

❖ **Solution:**

$$\frac{n}{\ln n} < \pi(n) < \frac{n}{\ln n - 1.08366}$$

$$\frac{1,000,000}{\ln(1,000,000)} < \pi(1,000,000) < \frac{1,000,000}{\ln(1,000,000) - 1.08366}$$

$$72,382 < \pi(1,000,000) < 78,543$$

➤ Hence, the number of primes less than 1,000,000 is **in the range** of (72,382,78,543). In fact, $\pi(1,000,000) = 78,498$ and **there are 78,498 primes**.

EULER'S
AND
FERMAT'S LITTLE THEOREMS

Preliminary – Coprime

- Recall that in Chapter 2, we learned that **two integers a and b** are **coprime** (or **relatively prime**) if their greatest common divisor is 1.

$$\gcd(a, b) = 1$$

- Note:
 1. 1 is relatively prime with any integer.
 2. If p is a **prime** number, then all the integers from 1 to $(p - 1)$ are relatively prime to p .

Euler's **Totient** Function, $\phi(n)$

- ❖ The **Euler's Totient Function** (or simply **Euler's Phi-Function**) $\phi(n)$ finds the number of integers that are smaller and coprime to a given integer n .
- ❖ Some **properties** of Euler's ϕ -Function:
 1. $\phi(1) = 0$.
 2. $\phi(p) = p - 1$, **if p is a prime**.
 3. $\phi(mn) = \phi(m) \times \phi(n)$, **if $\gcd(m, n) = 1$** .
 4. $\phi(p^e) = p^e - p^{e-1}$, **if p is a prime**.

Euler's **Totient** Function, $\phi(n)$

- From **Theorem 3**, any integer n can be decomposed as $n = p_1^{e_1} \cdot p_2^{e_2} \cdot p_3^{e_3} \cdots p_r^{e_r}$.

Then, we have:

$$\begin{aligned}\phi(n) &= \phi(p_1^{e_1} \cdot p_2^{e_2} \cdot p_3^{e_3} \cdots p_r^{e_r}) \\ &= \phi(p_1^{e_1}) \times \phi(p_2^{e_2}) \times \cdots \times \phi(p_r^{e_r}) \\ &= (p_1^{e_1} - p_1^{e_1-1})(p_2^{e_2} - p_2^{e_2-1}) \cdots (p_r^{e_r} - p_r^{e_r-1})\end{aligned}$$

Euler's **Totient** Function, $\phi(n)$

- **Examples:**

1. $\phi(13) = 13 - 1 = 12$. Since 13 is a **prime number**.
2. $\phi(100) = \phi(2^2 \cdot 5^2) = \phi(2^2) \times \phi(5^2) = (2^2 - 2^{2-1}) \times (5^2 - 5^{2-1}) = (2)(20) = 40$.

It indicates there are **40 numbers less than 100** that are **coprime to 100**.

- For a large value of n , computing $\phi(n)$ is possible only if its **prime factorization** can be found. In other words, the **hardness of finding $\phi(n)$** relies on the **difficulty of finding the integer factorization of n** .

Euler's **Totient** Function, $\phi(n)$

- Common mistake in finding $\phi(n)$.
- $\phi(49) = \phi(7) \times \phi(7) = (7 - 1)(7 - 1) = (6)(6) = 36$. This is **INCORRECT**, since it does **NOT** satisfy the property 3 such that $\gcd(7,7) \neq 1$. Instead,

$$\phi(49) = \phi(7^2) = 7^2 - 7^{2-1} = 49 - 7 = 42$$

Fermat's Little Theorem

❖ Theorem 4:

If p is a **prime** and a is a positive integer such that $a \nmid p$, then

$$a^{p-1} \equiv 1 \pmod{p}.$$

❖ Corollary 5:

If p is a **prime** and a is a positive integer, then

$$a^p \equiv a \pmod{p}.$$

Fermat's Little Theorem

- The Fermat's Little Theorem plays a very important role in **number theory**, and especially in **cryptography**, in which it helps to quickly finding a solution to some **exponentiation operations**.

❖ Examples:

1. $6^{10} \pmod{11} \equiv 1.$
2. $3^{18} \pmod{17} \equiv 3^2 \cdot 3^{16} \pmod{17} \equiv 9.$
3. **What is $10^{30} \pmod{29}$?**

Fermat's Little Theorem

- Besides speeding up the exponentiation operations, the Fermat's Little Theorem can be used to find **a multiplicative inverse** quickly if the **modulus is a prime p** , without having to use the Extended Euclidean algorithm.

❖ Corollary 6:

If p is a **prime** and a is a positive integer such that $a \nmid p$, then

$$a^{p-2} \equiv a^{-1} \pmod{p}.$$

Fermat's Little Theorem

❖ Examples:

1. $8^{-1} \pmod{17} \equiv 8^{17-2} \equiv 8^{15} \equiv 15 \pmod{17}.$

2. $5^{-1} \pmod{23} \equiv 5^{23-2} \equiv 5^{21} \equiv 14 \pmod{23}.$

3. $60^{-1} \pmod{101} \equiv 60^{101-2} \equiv 60^{99} \equiv 32 \pmod{101}.$

4. $22^{-1} \pmod{211} \equiv 22^{211-2} \equiv 22^{209} \equiv 48 \pmod{211}.$

✓ **Note:** These examples do **NOT** consider manual computation, instead it is with respect to computing devices.

Euler's Theorem

- Observe that in **Fermat's Little Theorem**, the modulus considered are all **prime** numbers. In general, such a specific requirement may not be the case.
- Euler modified Fermat's Little Theorem further into a generalized form - the **Euler's Theorem**, considering any positive modulus n (either prime or composite).

❖ Theorem 7:

If integers a and n are **coprime** (relatively prime), then

$$a^{\phi(n)} \equiv 1 \pmod{n}.$$

Euler's Theorem

- Similar to Fermat's Little Theorem, the Euler's Theorem is useful too in **speeding up the exponentiation operation.**

❖ Examples:

1. $6^{24} \pmod{35} \equiv 1$. Since $\phi(35) = (4)(6) = 24$
2. $20^{62} \pmod{77} \equiv 20^2 \cdot 20^{60} \pmod{17} \equiv 15$. Since $\phi(77) = (6)(10) = 60$
3. **What is $7^{26} \pmod{36}$?**

Euler's Theorem

- Besides, it also **finds the multiplicative inverses** without the need of the Extended Euclidean algorithm via modifying the theorem slightly.

❖ Corollary 8:

If integers a and n are **coprime** (relatively prime), then

$$a^{\phi(n)-1} \equiv a^{-1} \pmod{n}.$$

Euler's Theorem

❖ Examples:

1. $8^{-1} \pmod{77} \equiv 8^{\phi(77)-1} \equiv 8^{59} \equiv 29 \pmod{77}.$

2. $7^{-1} \pmod{15} \equiv 7^{\phi(15)-1} \equiv 7^7 \equiv 13 \pmod{15}.$

3. $60^{-1} \pmod{187} \equiv 60^{\phi(187)-1} \equiv 60^{159} \equiv 53 \pmod{187}.$

4. $71^{-1} \pmod{100} \equiv 71^{\phi(100)-1} \equiv 71^{39} \equiv 31 \pmod{100}.$

✓ **Note:** Again, these examples do **NOT** consider manual computation, instead it is with respect to computing devices.

PRIME NUMBERS AND PRIMALITY TESTING

Prime Numbers

- Generating **large prime numbers** that is suitable for cryptographic use is **NOT** a trivial task, since for a sufficient large numbers, **determining whether it is a PRIME is difficult**.
- For example, there are **two special types** of prime numbers that can be generated using special formulas:
 1. **Mersenne prime**: $M_p = 2^p - 1$, where p is some known primes.
 2. **Fermat prime**: $F_n = 2^{2^n} + 1$, where n is some non-negative integers.

Prime Numbers – Mersenne Primes

- Having the special formula of $M_p = 2^p - 1$, where p is some known primes, this structure however **DOES NOT** guarantee the output is a prime.

1. $M_2 = 2^2 - 1 = 3$ is a prime. 
2. $M_3 = 2^3 - 1 = 7$ is a prime. 
3. $M_5 = 2^5 - 1 = 31$ is a prime. 
4. $M_7 = 2^7 - 1 = 127$ is a prime. 
5. $M_{11} = 2^{11} - 1 = 2047 = (23)(89)$ is **NOT** a prime.  
6. $M_{13} = 2^{13} - 1 = 8191$ is a prime. 

Prime Numbers – Fermat Primes

- Having the special formula of $F_n = 2^{2^n} + 1$, where n is some non-negative integers, this structure also **DOES NOT** guarantee the output is a prime.

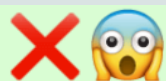
1. $F_0 = 2^{2^0} + 1 = 3$ is a prime. 

2. $F_1 = 2^{2^1} + 1 = 5$ is a prime. 

3. $F_2 = 2^{2^2} + 1 = 17$ is a prime. 

4. $F_3 = 2^{2^3} + 1 = 257$ is a prime. 

5. $F_4 = 2^{2^4} + 1 = 65537$ is a prime. 

6. $F_5 = 2^{2^5} + 1 = 4294967297 = (641)(6700417)$ is **NOT** a prime. 

Prime Numbers and Primality Testing

- There are **several methods** that can be used to test whether a given number is a prime, or composite, that categorized into two major branches.
 1. **Deterministic Algorithms**: Algorithms that accept an integer and **always** output either PRIME, or COMPOSITE.
 1. Sieve of Eratosthenes
 2. Divisibility Algorithm
 3. AKS Algorithm
 2. **Probabilistic Algorithms**: Algorithms that **DO NOT** guarantee the **correctness** of the result.
 1. Fermat Primality Test
 2. Miller-Rabin Test

Deterministic Algorithms – Sieve of Eratosthenes

- This method was introduced by Eratosthenes, a Greek mathematician to find all the primes less than a given number n .
- 1. Create a list of consecutive integers from 2 through n .
- 2. Initially, let $p = 2$ (the **smallest prime**). Enumerate the multiples of p and sieve through to n (these will be $2p, 3p, 4p, \dots$).
- 3. Find the smallest number in the list greater than p that is **not marked**. If there was no such number, stop. Otherwise, p now equal this new number (the next prime) and repeat from Step 2.
- 4. On termination, **the numbers remaining not marked** in the list are **all the primes below n** .

Deterministic Algorithms – Sieve of Eratosthenes

Example 1: The **primes between 1 and 100** through the sieve of Eratosthenes.

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Deterministic Algorithms – Sieve of Eratosthenes

list of prime numbers.

Example 1: The **primes between 1 and 100** through the sieve of Eratosthenes.

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Deterministic Algorithms – Sieve of Eratosthenes

- **Exercise 1**: Determine the **primes between 101 and 150** through the sieve of Eratosthenes.

101	102	103	104	105	106	107	108	109	110
111	112	113	114	115	116	117	118	119	120
121	122	123	124	125	126	127	128	129	130
131	132	133	134	135	136	137	138	139	140
141	142	143	144	145	146	147	148	149	150

Deterministic Algorithms – Sieve of Eratosthenes

- **Exercise 1:** Determine the **primes between 101 and 150** through the sieve of Eratosthenes.

101	102	103	104	105	106	107	108	109	110
111	112	113	114	115	116	117	118	119	120
121	122	123	124	125	126	127	128	129	130
131	132	133	134	135	136	137	138	139	140
141	142	143	144	145	146	147	148	149	150

Deterministic Algorithms – Divisibility Algorithm

- The divisibility algorithm (or divisibility test) determines if a number n is prime by testing if all the **primes less than $\sqrt{n}$** divides it completely, or otherwise.
1. Given the integer n , find $\sqrt{n}$.
 2. List all the primes less than $\sqrt{n}$, use these primes to divide n .
 3. If all the listed primes **DOES NOT** divide n , then n is a prime. Otherwise, n is composite.

Deterministic Algorithms – Divisibility Algorithm

- The bit-operation complexity of the divisibility algorithm is exponential. That is, given an n_b -bit integer, the algorithm runs at $\mathcal{O}(2^{\frac{n_b}{2}})$ complexity.

Algorithm 9.1 *Pseudocode for the divisibility test*

```
Divisibility_Test ( $n$ )           //  $n$  is the number to test for primality
{
   $r \leftarrow 2$ 
  while ( $r < \sqrt{n}$ )
  {
    if ( $r \mid n$ ) return "a composite"
     $r \leftarrow r + 1$ 
  }
  return "a prime"
}
```

Deterministic Algorithms – Divisibility Algorithm

Example 2: Determine if 349 is a prime.

Solution: Since $\sqrt{349} = 18.6 \dots$, all the primes less than $\sqrt{349}$ are 2, 3, 5, 7, 11, 13 and 17. We check if all these primes divides 349.

❖ **Conclusion:** Since all the primes does not divide 349, hence **349 is a prime**.

Primes	Does it divides 349
2	No
3	No
5	No
7	No
11	No
13	No
17	No

Deterministic Algorithms – AKS Algorithm

- The **AKS algorithm** is an deterministic algorithm devised by **A**grawal, **K**ayal and **S**axena in 2002, which they claimed to be an algorithm used for primality testing with polynomial bit-operation time complexity of $\mathcal{O}((\log_2 n_b)^{12})$.
 - This algorithm utilized the fact of $(x - a)^n \equiv (x^n - a) \pmod{n}$ in determining whether an integer at n is a prime or not.
- ✓ **Note:** We will not learn the AKS algorithm due to its complex computation that **involves polynomial rings reduction**.

Probabilistic Algorithms – Fermat Primality Test

- Recall the **Fermat's Little Theorem (Theorem 4)**, that if an integer p is a prime, then $a^{p-1} \equiv 1 \pmod{p}$ for an integer a such that $a \nmid p$.
- If n is a prime number, then congruence holds. However, if the congruence holds, it **DOES NOT imply** that n is a prime.
- Based on this statement, we can now define the **Fermat Primality Test**.

If n is a **prime**, then $a^{n-1} \equiv 1 \pmod{n}$.

If n is a **composite**, then it is **possible** that $a^{n-1} \equiv 1 \pmod{n}$.

Probabilistic Algorithms – Fermat Primality Test

- In other words, a **prime certainly passes** the Fermat Primality test. But a **composite may pass** the same test with some probability ε .
- ❖ **Example 3:** Determine if the integer 561 is a prime.
- ❖ **Solution:**
 1. Choose a positive integer $a = 2$, we notice that $2^{561-1} \equiv 1 \pmod{561}$ which passes the Fermat Primality test. However, $561 = 17 \times 33$ which is a composite.
 2. Choose another positive integer $a = 3$, we have $3^{561-1} \equiv 375 \pmod{561}$. Hence, 561 is NOT a prime.

Probabilistic Algorithms – Miller–Rabin Test

- The **Miller-Rabin test** combines both the **Fermat Primality test** and **Square-root test** (not covered in this course) in a very elegant technique to find a **strong pseudoprime** (a prime with a very high probability).
- This test expresses the integer $n - 1$ as a **product of an odd number m and power of 2**. That is, given any integer n ,

$$n - 1 = m \times 2^k$$

Probabilistic Algorithms – Miller–Rabin Test

- This expression is then undergone the **Fermat Primality test** using a positive integer base a such that $a \nmid n$.

$$a^{n-1} = a^{m \times 2^k} = [a^m]^{2^k} = [a^m]^{\overbrace{2 \cdot 2 \cdot \dots \cdot 2}^{k \text{ times}}}$$

- Instead of computing $a^{n-1} \equiv 1 \pmod{n}$ in one step, we perform $k + 1$ computations, with each computation that a **square-root test** is included.

Probabilistic Algorithms – Miller–Rabin Test

Miller_Rabin_Test (n, a)

// n is the number; a is the base.

{

Find m and k such that $n - 1 = m \times 2^k$

$T \leftarrow a^m \bmod n$

if ($T = \pm 1$) return "*a prime*"

for ($i \leftarrow 1$ to $k - 1$)

// $k - 1$ is the maximum number of steps.

{

$T \leftarrow T^2 \bmod n$

if ($T = +1$) return "*a composite*"

if ($T = -1$) return "*a prime*"

}

return "*a composite*"

}

Probabilistic Algorithms – Miller–Rabin Test

- **Example 4:** Using Miller-Rabin test, with base $a = 2$, determine whether the integer 561 is a prime.
- **Solution:** Since $561 - 1 = 35 \times 2^4$, we have $m = 35$ and $k = 4$. Via Miller-Rabin algorithm from $k = 0$ to $k = 3$, we found that 561 is a composite.

$k = 0$	$T \equiv 2^{35} \pmod{561} \equiv 263$	
$k = 1$	$T \equiv 263^2 \pmod{561} \equiv 166$	
$k = 2$	$T \equiv 166^2 \pmod{561} \equiv 67$	
$k = 3$	$T \equiv 67^2 \pmod{561} \equiv +1$	Composite

- ❖ The number $a = 2$ that proved **561 is a composite** is called a '**witness**'.

Probabilistic Algorithms – Miller–Rabin Test

- **Example 5:** Using Miller-Rabin test, with base $a = 2$, determine whether the integer 49 is a prime.
- **Solution:** Since $49 - 1 = 3 \times 2^4$, we have $m = 3$ and $k = 4$. Via Miller-Rabin algorithm from $k = 0$ to $k = 3$, we found that 49 is a composite.

$k = 0$	$T \equiv 2^3 \pmod{49} \equiv 8$	
$k = 1$	$T \equiv 8^2 \pmod{49} \equiv 15$	
$k = 2$	$T \equiv 15^2 \pmod{49} \equiv 29$	
$k = 3$	$T \equiv 29^2 \pmod{49} \equiv 8$ (failed)	Composite

❖ Hence, **49 is a composite** with **witness** $a = 2$.

Probabilistic Algorithms – Miller–Rabin Test

- **Example 6:** Using Miller-Rabin test, with base $a = 2$, determine whether the integer 349 is a prime.
- **Solution:** Since $349 - 1 = 87 \times 2^2$, we have $m = 87$ and $k = 2$. Via Miller-Rabin algorithm from $k = 0$ to $k = 1$:

$k = 0$	$T \equiv 2^{87} \pmod{349} \equiv 213$	
$k = 1$	$T \equiv 213^2 \pmod{349} \equiv 348 \equiv -1$	Prime

❖ Hence, **349 is a prime number.**

Primality Testing – The Recommended Test

- Currently, the most popular primality test used in cryptography is the **combination** of **divisibility test** and **Miller-Rabin test**.
- 1. Given a large odd integer n , perform divisibility test by dividing some of the primes less than $\sqrt{n}$.
- 2. If the number n passes the divisibility test **for some reason**, perform Miller-Rabin test using a set of bases a . Sometimes it requires more bases to provide sufficient 'proof'.
- 3. If this number n **passes** the Miller-Rabin with sufficiently many bases a , then we can conclude that the integer n is a **strong pseudoprime**.

Primality Testing – The Recommended Test

- **Example 7:** Determine whether the integer 4033 is a prime.
- **Solution:** Although it is easy to verify that 4033 is actually a composite of 37×109 , let's assume that we do not know it initially.
 1. Using divisibility test, $\sqrt{4033} = 63.5 \dots$ So, we divide 4033 using primes $n = 2, 3, 5, 7, 11, 13, 17, 19, 23$ but none of them works (assume that we are somehow lazy to test all the primes less than 63 for some reasons).
 2. Next, we use Miller-Rabin test with base $a = 2$. Since $4033 - 1 = 63 \times 2^6$, we have $m = 63$ and $k = 6$.

Primality Testing – The Recommended Test

$k = 0$	$T \equiv 2^{63} \pmod{4033} \equiv 3521$	
$k = 1$	$T \equiv 3521^2 \pmod{4033} \equiv 4032 \equiv -1$	Pass, but prime?

3. The first base $a = 2$ tells us it is a prime. But let's say we are NOT satisfied with the result, we choose the next base $a = 3$ for the second round of Miller-Rabin test.

$k = 0$	$T \equiv 3^{63} \pmod{4033} \equiv 3551$	
$k = 1$	$T \equiv 3551^2 \pmod{4033} \equiv 2443$	
$k = 2$	$T \equiv 2443^2 \pmod{4033} \equiv 3442$	
$k = 3$	$T \equiv 3551^2 \pmod{4033} \equiv 2443$	
$k = 4$	$T \equiv 2443^2 \pmod{4033} \equiv 3442$	
$k = 5$	$T \equiv 3551^2 \pmod{4033} \equiv 2443$	Composite

Primality Testing – The Recommended Test

4. Hence, the base $a = 3$ tells us that it fails the Miller-Rabin test. It is a witness that proves 4033 is a composite.
-
- **Note:** What we simulated here is a small odd number. For a very large number, such as 998,761 and 7,622,296,651, or even larger than these, we can only hope that it is indeed primes, or if not, a **strong pseudoprimes**.

Primality Testing

- **Exercise 2:** Using the Miller-Rabin test, determine whether the integer 6899 is a prime number.
- **Solution:** We use different bases here. Since $6899 - 1 = 3449 \times 2$, we have $m = 3499$ and $k = 1$. Hence, we only do the Miller-Rabin iteration once.
- The integer 6899 passes the Miller-Rabin test with 4 different bases. **We can conclude that 6899 is a strong pseudoprime (In fact, 6899 is a prime).**

Bases	Iteration	Computation	Decision
$a = 2$	$k = 0$	$T \equiv 2^{3449} \pmod{6899} \equiv 6898 \equiv -1$	Prime
$a = 3$	$k = 0$	$T \equiv 3^{3449} \pmod{6899} \equiv +1$	Prime
$a = 5$	$k = 0$	$T \equiv 5^{3449} \pmod{6899} \equiv +1$	Prime
$a = 7$	$k = 0$	$T \equiv 7^{3449} \pmod{6899} \equiv 6898 \equiv -1$	Prime

CHINESE REMAINDER THEOREM (Optional)

Chinese Remainder Theorem

- The **Chinese Remainder Theorem (CRT)** is used to solve a set of congruent equations with **single variable** but **multiples moduli** that are **relatively prime**:

$$x \equiv a_1 \pmod{m_1}$$

$$x \equiv a_2 \pmod{m_2}$$

...

$$x \equiv a_k \pmod{m_k}$$

- In other words, if $\text{gcd}(m_1, m_2, \dots, m_k) = 1$, then the Chinese Remainder Theorem says that **there exists a unique solution** such that $x \equiv c \pmod{m_1 m_2 \cdots m_k}$.

Chinese Remainder Theorem

- The CRT is believed to have been discovered by the Chinese Mathematician **Sun-Tzu** in around 100 A.D., which was how the term '**Chinese**' appeared.
- The **original problem** that gave birth to the CRT is as follows:

There are certain things whose number is unknown. If we count them by threes, we have two left over; by fives, we have three left over; and by sevens, two are left over.

How many things are there?

Chinese Remainder Theorem

- If we formulate the original problem mathematical, it requires us to find a unique solution x such that:

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 2 \pmod{7}$$

- The **solution** to this set of congruent equations is $x \equiv 23 \pmod{105}$, which we will learn on how to compute them in the subsequent slides.

Chinese Remainder Theorem

- **Steps to find the solution to the congruent equations via the CRT:**
 1. Compute $M = m_1 m_2 \cdots m_k$. This is the common moduli.
 2. Compute $M_1 = \frac{M}{m_1}, M_2 = \frac{M}{m_2}, \dots, M_k = \frac{M}{m_k}$.
 3. Compute the **multiplicative inverses** of $M_1, M_2, \dots, M_k$ using the corresponding moduli $m_1, m_2, \dots, m_k$ via the **Extended Euclidean algorithm** (if necessary).
 4. The solution to the simultaneous congruent equations is

$$x \equiv (a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1} + \cdots + a_k M_k M_k^{-1}) \pmod{M}$$

Chinese Remainder Theorem

❖ **Example 8:** Referring to the original problem, find the unique solution to the system of congruent equations:

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 2 \pmod{7}$$

❖ **Solution:**

1. Compute $M = 3 \times 5 \times 7 = 105$

Chinese Remainder Theorem

2. Compute $M_1 = \frac{105}{3} = 35$, $M_2 = \frac{105}{5} = 21$, $M_3 = \frac{105}{7} = 15$.
3. Compute each multiplicative inverses $M_1^{-1} \equiv 2 \pmod{3}$, $M_2^{-1} \equiv 1 \pmod{5}$, $M_3^{-1} \equiv 1 \pmod{7}$.
4. Compute the solution to the congruent equations:
$$x \equiv (2 \cdot 35 \cdot 2 + 3 \cdot 21 \cdot 1 + 2 \cdot 15 \cdot 1) \pmod{105} \equiv 233 \equiv \mathbf{23 \pmod{105}}$$

Chinese Remainder Theorem

- **Why is the Chinese Remainder Theorem important in Cryptography?**
 1. It can be used to find the **quadratic congruence** in the form of $x^2 \equiv a \pmod{n}$.
 2. It is used in designing the **RSA-CRT cryptosystem**. An RSA variant that uses multi private keys in encryption, and signature schemes.
 3. It can also be used in cryptanalyzing the **Multi-prime RSA** cryptosystem under suitable condition.

Chinese Remainder Theorem

- **Exercise 3:** Using the Chinese Remainder Theorem, find a solution to the system of congruent equations of:

$$x \equiv 3 \pmod{7}$$

$$x \equiv 3 \pmod{13}$$

$$x \equiv 0 \pmod{12}$$

FAST EXPONENTIATION ALGORITHM

Fast Exponentiation Algorithm

- Basic Mathematics about **Exponentiation** and **Logarithm**.

- Exponentiation and Logarithm are **inverses** of each other.

$$y = a^x \iff x = \log_a y$$

- In cryptography, we are interested in the **modular exponentiation**, which utilized in the RSA cryptosystem in both the encryption and decryption operations, where:

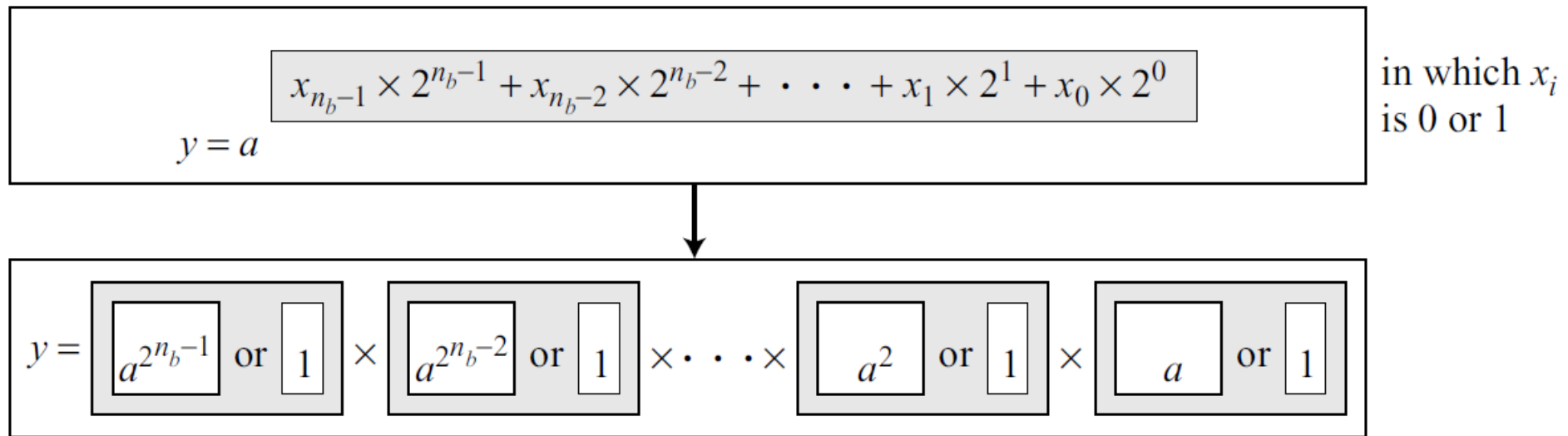
$$y = a^x \pmod{n}$$

Fast Exponentiation Algorithm

- Unfortunately, there is **no efficient way** to perform such an operation in most computer languages, especially when the **exponent is very large** (says 1024 or 2048 bits).
- Hence, the **Fast Exponentiation Algorithm** is devised to provide an efficient way to complete this task. It uses the **square-and-multiply** technique by first converting the exponent into binary digits of n_b bits (from x_0 to x_{n_b-1}).
- **Example:** $11 = (1011)_2$. Then $a^{11} = a^{(1011)_2} = a^8 \times 1 \times a^2 \times a^1$

Fast Exponentiation Algorithm

Figure 9.6 *The idea behind the square-and-multiply method*



Example:

$$y = a^9 = a^{1001_2} = a^8 \times 1 \times 1 \times a$$

Fast Exponentiation Algorithm

Algorithm 9.7 *Pseudocode for square-and-multiply algorithm*

Square_and_Multiply (a, x, n)

{

$y \leftarrow 1$

 for ($i \leftarrow 0$ to $n_b - 1$)

 // n_b is the number of bits in x

 {

 if ($x_i = 1$) $y \leftarrow a \times y \bmod n$

 // multiply only if the bit is 1

$a \leftarrow a^2 \bmod n$

 // squaring is not needed in the last iteration

 }

 return y

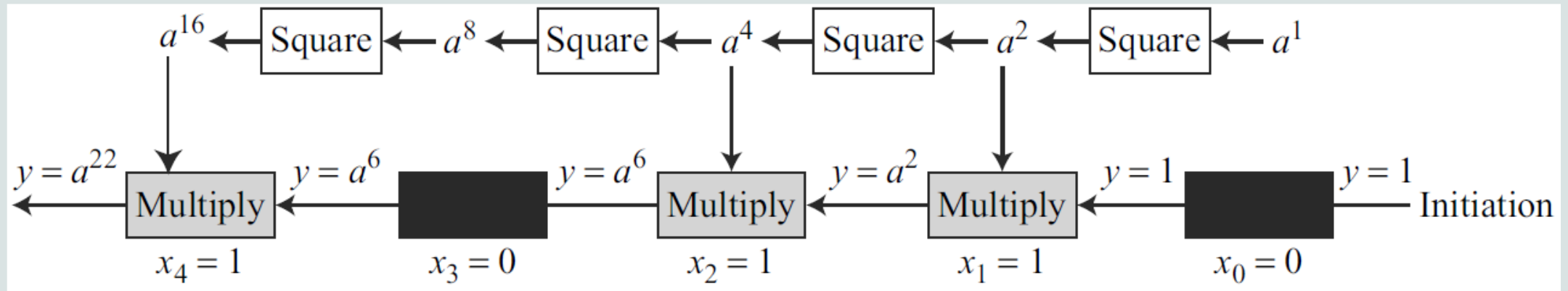
}

Fast Exponentiation Algorithm

- The main idea behind the **square-and-multiply** technique:
 - The technique starts with the rightmost bit (Least Significant Bit, LSB).
 1. If the bit $n_b = 0$, then **only squaring operation** is performed.
 2. If the bit $n_b = 1$, then **squaring operation** is performed, followed by **multiplication** operation.
- The fast exponentiation algorithm runs in the complexity of $\mathcal{O}(n_b)$, which is polynomial (efficient).

Fast Exponentiation Algorithm

- **Example 9:** Find the modular exponentiation of $17^{22} \pmod{21}$ using the fast exponentiation algorithm.
- **Solution:** The exponent $22 = (10110)_2$, hence the square-and-multiply processes are



Fast Exponentiation Algorithm

i	x_i	<i>Multiplication</i> (Initialization: $y = 1$)	<i>Squaring</i> (Initialization: $a = 17$)
0	0	$\rightarrow$	$a = 17^2 \bmod 21 = 16$
1	1	$y = 1 \times 16 \bmod 21 = 16 \rightarrow$	$a = 16^2 \bmod 21 = 4$
2	1	$y = 16 \times 4 \bmod 21 = 1 \rightarrow$	$a = 4^2 \bmod 21 = 16$
3	0	$\rightarrow$	$a = 16^2 \bmod 21 = 4$
4	1	$y = 1 \times 4 \bmod 21 = 4 \rightarrow$	

❖ Hence, $17^{22} \bmod 21 \equiv 4$.

Fast Exponentiation Algorithm

9. Find the results of the following using the square-and-multiply method of fast exponentiation algorithm.

(i) $320^{23} \bmod 461$

- Since $23 = (10111)_2$, using fast exponentiation algorithm, we compute and found $320^{23} \bmod 461 \equiv 373$.

x_i	Multiplication	Squaring
1	$1 \cdot 320 \equiv 320 \pmod{461}$	$320^2 \equiv 58 \pmod{461}$
1	$320 \cdot 58 \equiv 120 \pmod{461}$	$58^2 \equiv 137 \pmod{461}$
1	$120 \cdot 137 \equiv 305 \pmod{461}$	$137^2 \equiv 329 \pmod{461}$
0		$329^2 \equiv 367 \pmod{461}$
1	$305 \cdot 367 \equiv 373 \pmod{461}$	

Main References for Chapter 7

1. Forouzan, B.A. "***Introduction to Cryptography and Network Security***", Third Edition, McGraw-Hill, 2008.
2. Stallings, W. "***Cryptography and Network Security: Principles and Practices***", Fifth Edition, Pearson Prentice Hall, 2011.

~ THE END ☺ ~